



Systeme centralisé de gestion et d'analyse de logs applicatifs

Soutenance — Cours BD NoSQL

Groupe 3 — LogTech Solutions

Université Gamal Abdel Nasser de Conakry — L3 Développement Logiciel

Encadrant : Mr. Djiba Kaba

Mai 2026

Plan de la présentation

- | | | |
|---|-------------------------------------|--|
| 1 | Contexte & problématique | <i>Pourquoi LogWatch à l'UGANC</i> |
| 2 | Solution & architecture | <i>Stack technique et modèle 3 couches</i> |
| 3 | Modélisation MongoDB | <i>3 collections, schéma flexible</i> |
| 4 | Pipelines d'agrégation | <i>4 pipelines analytiques</i> |
| 5 | Performance & index | <i>Mesures explain() réelles</i> |
| 6 | Démonstration live | <i>Le dashboard en action</i> |
| 7 | Bilan & perspectives | <i>Contraintes CDC ✓ et conclusion</i> |

1. Contexte & problématique

L'infrastructure informatique universitaire

Situation actuelle

- Plus de 10 applications hétérogènes
- 4 technologies différentes : Java, Node.js, Python, PHP
- Chaque application génère ses propres logs
- Aucune centralisation
- Diagnostic manuel, fastidieux, lent

Conséquences

- Détection tardive des incidents
- Pas de vue d'ensemble sur la santé du SI
- Difficultés à corréler les erreurs entre apps
- Aucune détection d'anomalie automatisée
- Audit sécurité quasi impossible

→ **Besoin d'un système centralisé temps réel pour superviser tout le SI**

2. Solution — Pourquoi MongoDB ?

SQL vs NoSQL pour des logs hétérogènes

SQL classique

Schéma fixe — colonnes pré-définies

- 4 tables différentes (Java, Web, Sec, DB)
- Beaucoup de colonnes NULL
- JOIN multiples pour analyser
- Migration lourde si nouveau type
- Performance dégradée à grand volume

MongoDB

Schéma flexible — strict: false

- 1 collection logs — tous les types ensemble
- Champs spécifiques par document
- \$exists pour filtrer par type
- Nouveau type ajouté sans migration
- Pipelines d'agrégation natifs

3. Architecture LogWatch

Modèle 3 couches — Frontend / Backend / Base

FRONTEND

HTML / CSS / JavaScript vanilla • Chart.js 4.x • Phosphor Icons

Dashboard temps réel, 5 pages, modal détail, auto-refresh



BACKEND

Node.js 18+ • Express 5.x • Mongoose 9.x • Helmet + Morgan + CORS

26 routes REST testées à 100% — pattern MVC (routes / controllers / models)



BASE DE DONNÉES

MongoDB Atlas (cloud, M0 Free Tier) • Cluster Projet3-LogWatch

3 collections — 2 013 logs, 10 applications, 67 alertes — 3 index actifs

4. Modélisation MongoDB

3 collections — relation par référence (app_id)

applications

10 documents

- app_id (PK)
- nom, version
- environnement
- technologie
- responsable
- sla_pct

logs

2 013 documents

- log_id (PK)
- app_id (FK)
- timestamp
- level
- message
- + champs flexibles

alertes_systeme

67 documents

- alerte_id (PK)
- app_id (FK)
- timestamp
- type_alerte
- resolue
- assignee_uid

Choix : référence (app_id) plutôt qu'imbrication — 2 000+ logs par app, imbrication non viable

Schéma flexible — preuve par l'exemple

Deux documents dans la même collection logs — structures totalement différentes

Log Java (erreur)

```
{
  log_id: "LOG-JAVA-000042",
  app_id: "app_si_etudiant",
  timestamp: ISODate(...),
  level: "CRITICAL",
  message: "NullPointerException...",
  source_fichier: "UserService.java",
  ligne_code: 287,
  exception_type: "NullPointerException...",
  stack_trace: "java.lang...",
  nb_occurrences: 12
}
```

Log Web

```
{
  log_id: "LOG-WEB-000156",
  app_id: "WEB",
  timestamp: ISODate(...),
  level: "INFO",
  message: "GET /api/...",
  methode_http: "GET",
  url: "/api/etudiants",
  code_statut: 200,
  duree_ms: 145,
  ip_source: "10.0.2.34"
}
```

En SQL : 2 tables avec des colonnes différentes + JOIN ou colonne NULL partout

5. Pipelines d'agrégation MongoDB

4 pipelines analytiques — opérateurs `$group`, `$match`, `$project`, `$sort`, `$cond`

1

Taux d'erreur par application

Quel SI est le plus instable ?

SQL équivalent

GROUP BY + CASE WHEN

2

Top 10 erreurs fréquentes

Quels messages reviennent ?

SQL équivalent

GROUP BY message ORDER BY count

3

Distribution temporelle

À quelles heures les pics ?

SQL équivalent

GROUP BY HOUR(timestamp)

4

Détection d'anomalies

Pics anormaux > seuil

SQL équivalent

GROUP BY + HAVING COUNT > 3

6. Performance — IXSCAN vs COLLSCAN

Mesures explain() réelles sur le cluster Atlas (2 013 logs)

Scénario	Stratégie	Docs examinés	Docs retournés	Gain
Filtre level=ERROR AVEC index	IXSCAN ✓	457	457	x4.4
Filtre level=ERROR SANS index	COLLSCAN ✗	2 013	457	baseline
Filtre timestamp 24h AVEC index	IXSCAN ✓	11	11	x183

Projection production

Sur 10 M de logs : COLLSCAN $\approx$ 10 s $\rightarrow$ IXSCAN $\approx$ 2.3 s (gain critique pour un dashboard temps réel)

7. Démonstration live

Ce que nous allons montrer pendant 10 minutes

1

Dashboard temps réel

KPIs, graphiques, auto-refresh 30s, mode clair/sombre

2

Insertion d'un log

Formulaire dynamique selon le type — apparition instantanée

3

Recherche avancée

\$regex sur message, \$in pour level, filtre temporel

4

Filtres par type (\$exists)

Logs Java, Web, Sécurité, DB lents — un clic chacun

5

Pipelines visualisés

Les 4 agrégations affichées avec graphiques

6

Audit sécurité

IPs suspectes, utilisateurs avec > 5 tentatives échouées

Bilan & difficultés rencontrées

Toutes les contraintes du cahier des charges sont remplies

Contraintes CDC ✓

- Volume 2 000+ logs
- 4 sources distinctes (Java/Web/Sec/DB)
- \$exists pour stack_trace
- 4 pipelines d'agrégation
- Pipeline anomalies (\$group + HAVING)
- Index level + timestamp + app_id
- Insertion temps réel via UI
- Application web fonctionnelle

Difficultés rencontrées

- Configuration IP MongoDB Atlas (whitelisting)
- Choix imbrication vs référence pour logs/apps
- Syntaxe MongoDB du HAVING SQL (\$match après \$group)
- Cohérence du schéma flexible (4 types)
- Coordination Git en équipe (commits, branches)
- Génération de données réalistes avec Faker

Merci pour votre attention

Questions & démonstration live

Le groupe est prêt à répondre individuellement sur :

**modélisation MongoDB • pipelines d'agrégation • index et performances •
architecture web et REST • comparaison SQL/NoSQL**